Deep Matching and Retrieval of Indic Script Documents

Bachelor's Thesis Project (BTP)

Pappu Meghana (S20230030400)

AI & Data Science | IIIT Sricity

Introduction

Roman Scripts

Simple left-to-right, one-dimensional structure. Near 1:1 mapping between characters and Unicode. Traditional string matching works well.

Indic Scripts

Multi-dimensional, hierarchical structure. Aksharas grow in multiple directions. Complex many-to-many mapping between visual glyphs and Unicode.

The Challenge

UTF-8 byte sequences don't reflect linguistic structure. Traditional LED treats code points as atomic units, missing structural relationships.

🔍 **Goal: Learn a structurally-aware similarity function for Indic Unicode sequences using deep learning**

Representation Levels

Three Levels of Representation — with Increasing Intuitiveness (→)

UTF-8

Internal Machine Representation

```
1110xxxx 10xxxxxx 10xxxxxx
```

Byte-level encoding. Multiple bytes per character. No linguistic meaning in structure.

Least Intuitive

→

Unicode

Code Point

```
0x0C15 (క)
0x0C4d (◌్)
0x0C37 (ష)
```

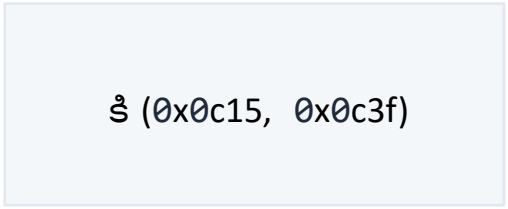
Code point sequences. Structured but linear. Fails to show multi-dimensional nature.

Moderate

→

Glyph

Visual Representation



Connected components capturing true visual structure. Most linguistically meaningful.

Most Intuitive

Problem Statement

Formal Definition

Given two Telugu words x_1 and x_2 represented as Unicode code-point sequences, learn a distance function:

$$\hat{y} = f(x_1, x_2) \quad \text{such that} \quad \hat{y} \approx \text{LED}(x_1, x_2)$$

① **Structural Complexity**

Aksharas are composite units — base consonant + vowel/consonant modifiers stacked multi-dimensionally. LED treats each code point atomically, missing structural weight.

② **Many-to-Many Mapping**

A single akshara maps to multiple Unicode code points, and the same Unicode sequence can represent different visual forms depending on context.

③ **Quadratic Complexity of LED**

LED has $O(|x_1| \cdot |x_2|)$ complexity. Our model reduces this to $O(|x_1|) + O(|x_2|)$ at inference by learning embeddings.

Previous Work (Last Presentation)

What was covered:

1

Problem Formulation

Defined the task: learn f(x1, x2) -> y where y = LED(x1, x2). Identified Indic script challenges — akshara complexity, many-to-many Unicode mapping, quadratic LED cost.

2

Literature Review

Navarro (2001): LED limitations. Bromley et al. (1993): Siamese networks. Bellet et al. (2015): Metric learning. Krishnan et al. (2015): Indic script encoding issues.

3

Generic Architecture Proposed

Proposed Unicode-level Siamese network with shared encoders. Two encoder pathways: RNN (context) and CNN (for spatial context). MSE regression loss on LED.

4

Initial Word Collection

Telugu words collected and mapped to Unicode sequences. Concept of word -> Unicode hex -> one-hot encoding pipeline described. Fixed vocab (60 code points) defined.

Present Work — Preprocessing Pipeline

Raw UTF-8 Pairs

db1_utf8_pairs.csv

122 word pairs
(Telugu UTF-8)

>

Unicode Sequences

db2_unicode_pairs.csv

122 pairs
(hex code points
+ LED labels)

>

Integer Mapping

db2_unicode_integers.csv

122 pairs
(decimal Unicode
integers)

>

Final Processed

db3_processed.csv

122 pairs
(vocab-indexed
sequences)

vocab.txt — 60 code points

60 unique Telugu Unicode code points selected after filtering rare elements from the 128-point block (0x0C00-0x0C7F). Each mapped to index 0-59 for integer encoding.

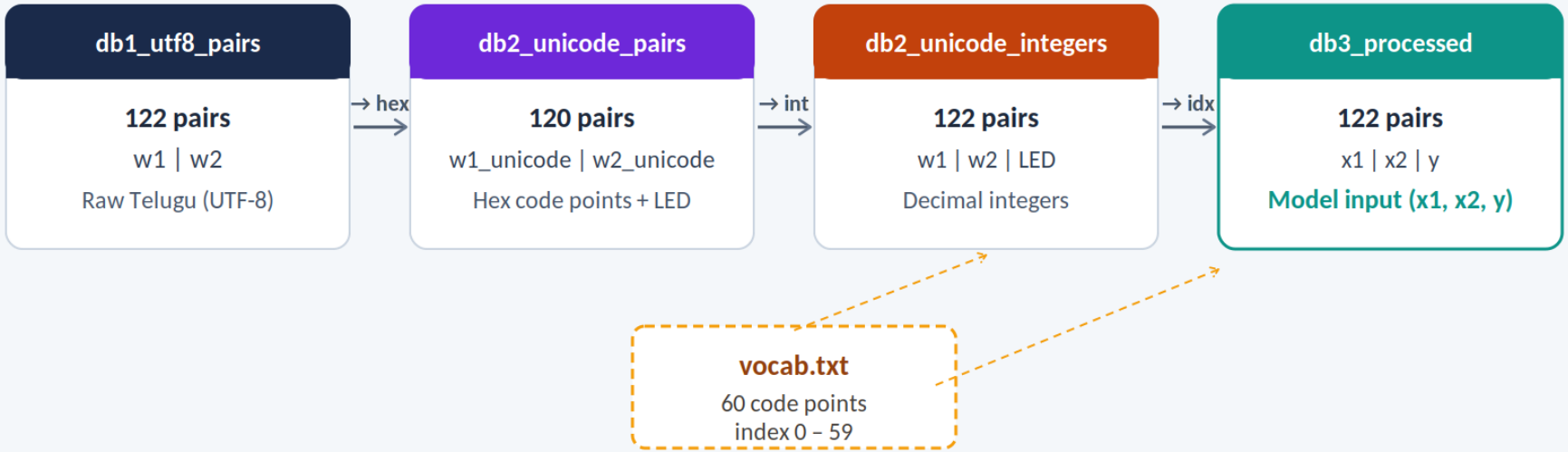
LED Distribution (122 pairs)

Range: 2-19 | Mean: 7.85 | Most common: LED=5 (20), LED=4 & 6 (15 each)
Covers short to long edit distances — good diversity for training.

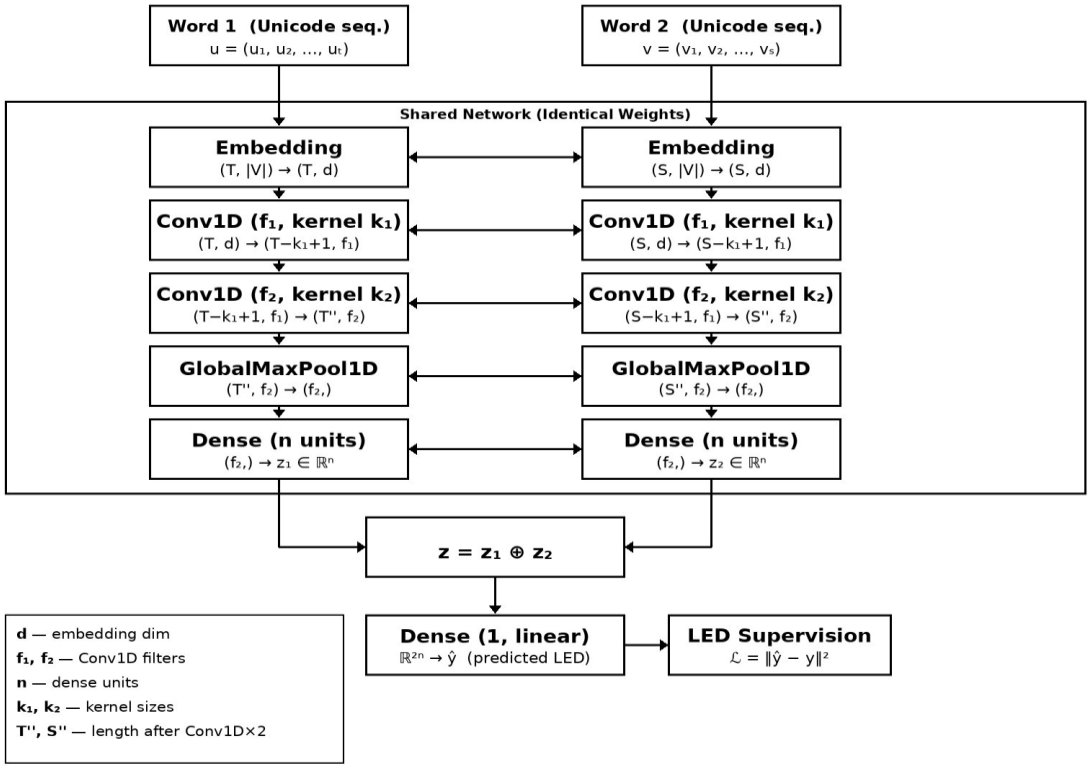
Data Collection & Dataset Creation

122	60	2–19	4 DBs
Telugu Word Pairs Collected	Vocab (Unicode code pts)	LED Range (Mean: 7.85)	Pipeline Stages

Dataset Creation Pipeline



Model Architecture — CNN-Based Siamese Network



Key Design Choices

Input:	Unicode seq. (1×128 one-hot per code point)
Embedding:	Maps to d-dim space (e.g. d=64)
Conv1D ×2:	Learns spatial patterns between code points
GlobalMaxPool:	Resolves variable-length input
Dense:	Projects to n-dim representation (e.g. n=250)
Merge $z_1 \oplus z_2$:	Concatenate both word vectors
Loss:	MSE: $L = \|\|\hat{y} - y\|\|^2$ (LED supervision)

Implementation — Preprocessing Complete

Model Dimensions (Phase 2 Target)

Input	`60 x 10 (variable len, one-hot 60)`
Embedding	`10 × 64 (d = 64)`
Conv1D (5 kernels, k=5)	`6 × 250 (T−k+1 = 6 dimensions)`
Conv1D (5 kernels)	`2 × 400 (stacked convolution)`
GlobalMaxPool1D	`400 × 1`
Dense	`250 (per word vector)`
Concatenate $z_1 \oplus z_2$	`250(If concat=>diff(-))`
Dense (1, linear)	`1 → predicted LED (ŷ)`

Preprocessing Status

- ✓ 122 Telugu word pairs collected (db1_utf8_pairs.csv)
- ✓ UTF-8 → Unicode hex sequence conversion (db2_unicode_pairs.csv)
- ✓ LED computed for each pair as ground truth label
- ✓ Hex → decimal integer mapping (db2_unicode_integers.csv)
- ✓ Vocab filtered: 60 code points from 128-pt block (vocab.txt)
- ✓ Vocab index mapping: code point → index 0-59
- ✓ Final (x1, x2, y) triples dataset built (db3_processed.csv)
- ○ Model training on db3 dataset (Phase 2)
- ○ Evaluation & retrieval benchmarking

Future Work & Roadmap

Phase 1
COMPLETE

- Vocabulary & preprocessing pipeline
- Pairwise dataset with LED labels
- Architecture design (CNN Siamese)

Phase 2
NEXT

- Learn embedding space (dim < 128)
- Replace one-hot with learned embeddings
- Train CNN Siamese network
- Evaluate MSE alignment with LED

Phase 3
PLANNED

- Explore comparison strategies (concat, dot product, cosine)
- Cluster analysis of learned representations
- Performance benchmarking vs baseline LED

Phase 4
PLANNED

- Expand vocabulary; add modifier characters
- Glyph-level matching (visual representation)
- Generalize to other Indic scripts (Kannada, Devanagari)

Progress: Phase 1/4 Complete

References

[1] A. Bellet, A. Habrard, M. Sebban. "A Survey on Metric Learning for Feature Vectors and Structured Data." IEEE TPAMI, vol. 37, no. 11, pp. 2172--2189, 2015.

[2] P. K. Perepu, "A semantic-based deep learning framework for mathematical expression retrieval," Knowledge and Information Systems, vol. 68, p. 115, 2026.

[3] A. Graves. "Supervised Sequence Labelling with Recurrent Neural Networks." Springer, 2009.

[4] P. K. Perepu, "OpenMP Implementation of Parallel Longest Common Subsequence Algorithm for Mathematical Expression Retrieval," Parallel Processing Letters, vol. 31, no. 2, p. 2150007, 2021.

[5] G. Navarro. "A Guided Tour to Approximate String Matching." ACM Computing Surveys, vol. 33, no. 1, pp. 31--88, 2001.

Thank You

Questions & Discussion

Pappu Meghana | S20230030400 | AI & Data Science | IIIT Sricity